Actividad de Aprendizaje
Semana 10

Indicaciones Generales:

1. El plagio se sanciona con la suspensión o expulsión del estudiante de la Universidad. Reglamento General de Estudios.
2. Sea cuidadoso con su redacción, la cual formará parte de su calificación (aspectos léxicos, sintácticos, semánticos).
3. Lea bien la pregunta o enunciado antes de proceder a su desarrollo, administre su tiempo eficazmente.

Actividad 2.

Chatbot RAG sobre documentos PDF con Gemini, MongoDB Atlas y Streamlit Cloud

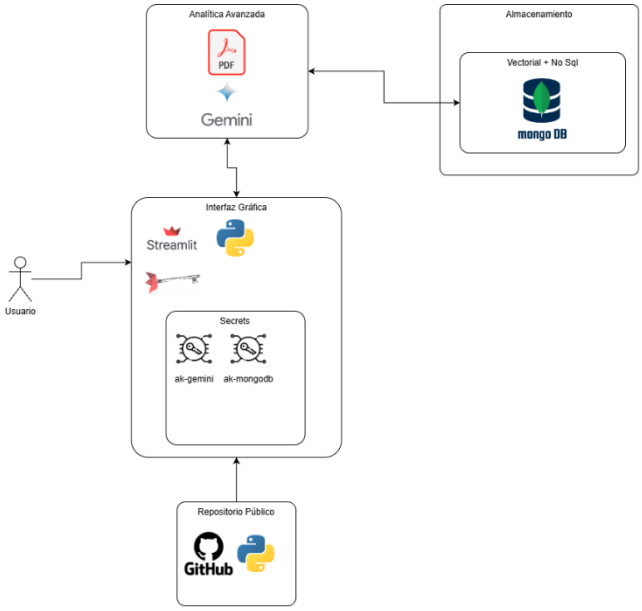
OBJETIVO:

Construir y desplegar un chatbot de Generación Aumentada por Recuperación (RAG) que responda preguntas sobre un documento PDF: extracción y fragmentación del texto, generación de embeddings con Gemini, almacenamiento y búsqueda vectorial en MongoDB Atlas, y publicación de una interfaz conversacional en Streamlit Community Cloud gestionando las credenciales mediante secretos.

¿QUÉ SE TRABAJARÁ EN LA ACTIVIDAD?

- Creación de un clúster gratuito en MongoDB Atlas.
- Lectura del PDF y fragmentación del texto en trozos (chunks) de 1000 caracteres.
- Generación de embeddings con gemini-embedding-001 (3072 dimensiones).
- Almacenamiento de fragmentos y embeddings en MongoDB Atlas y creación del índice vectorial.
- Recuperación semántica con $vectorSearch y generación de la respuesta con gemini-2.5-flash.
- Construcción de la interfaz de chat con Streamlit (app.py).
- Publicación del código en GitHub y despliegue en Streamlit Community Cloud usando secretos.

ARQUITECTURA DE LA SOLUCIÓN



Arquitectura del chatbot: interfaz en Streamlit con secretos, analítica con Gemini, almacenamiento vectorial en MongoDB Atlas y repositorio público en GitHub.

Componente	Tecnología	Función
Interfaz gráfica	Streamlit + Python	Recibe la pregunta del usuario y muestra la respuesta y los fragmentos recuperados.
Secrets	ak-gemini, ak-mongodb	Guardan GOOGLE_API_KEY y MONGODB_URI fuera del código, en la configuración de Streamlit Cloud.
Analítica avanzada	Gemini (embeddings + LLM)	Vectoriza el PDF y la pregunta; redacta la respuesta usando el contexto recuperado.
Almacenamiento	MongoDB Atlas (NoSQL + Vector Search)	Guarda cada fragmento con su embedding y resuelve la búsqueda por similitud.
Repositorio público	GitHub	Contiene app.py y requirements.txt; Streamlit Cloud despliega desde aquí.

CONCEPTOS

Concepto	Uso en la aplicación	Interpretación
RAG	Recuperar → generar	El LLM responde con fragmentos recuperados del documento y no solo con su conocimiento previo.
Chunking	Trozos de 1000 caracteres	Divide el texto en unidades pequeñas que se pueden vectorizar y recuperar.
Embedding	gemini-embedding-001	Vector numérico que representa el significado de un texto; textos similares quedan cercanos.
task_type	RETRIEVAL_DOCUMENT / RETRIEVAL_QUERY	Optimiza el embedding según sea un fragmento del documento o una consulta del usuario.
Índice vectorial	vector_index	Estructura de Atlas que permite buscar los vecinos más cercanos a un vector.
Secreto	st.secrets	Credencial almacenada fuera del código y del repositorio público.

INSTRUCCIONES:

Paso 1. Crear el clúster en MongoDB Atlas

Para crear el clúster utilice el manual **"MongoDB Atlas"** publicado en CANVAS. Al finalizar debe contar con un clúster gratuito (M0), un usuario de base de datos, acceso de red habilitado (0.0.0.0/0, para que Colab y Streamlit Cloud puedan conectarse) y la cadena de conexión:

```
mongodb+srv://<usuario>:<password>@<cluster>.mongodb.net/
```

Paso 2. Obtener la API key de Gemini y registrar los secretos en Colab

Genere una API key en Google AI Studio (https://aistudio.google.com/). En Colab, registre GOOGLE_API_KEY y MONGODB_URI en el panel de secretos (ícono de llave) y léalos con userdata; ninguna credencial debe quedar escrita en el notebook.

```
!pip install pymongo google-genai PyPDF2


from google.colab import userdata
GOOGLE_API_KEY = userdata.get('GOOGLE_API_KEY')
MONGODB_URI = userdata.get('MONGODB_URI')
```

Paso 3. Ingesta: conectar con Gemini y MongoDB

La sección Ingest del notebook app_llm.ipynb crea ambos clientes. La base de datos pdf_embeddings_db y la colección pdf_vectors se crean automáticamente con la primera inserción.

```
client_genai = genai.Client(api_key=GOOGLE_API_KEY)
client_mongo = pymongo.MongoClient(MONGODB_URI)
db = client_mongo.pdf_embeddings_db
collection = db.pdf_vectors
```

Paso 4. Ingesta: leer el PDF

leer_pdf recorre las páginas con PdfReader y concatena el texto extraído. La expresión "or """ evita errores en páginas sin texto (por ejemplo, páginas escaneadas).

```
def leer_pdf(path_pdf):
    reader = PdfReader(path_pdf)
    texto = ""
    for page in reader.pages:
        texto += (page.extract_text() or "") + "\n"
    return texto.strip()
```

Paso 5. Ingesta: generar embeddings

crear_embedding convierte un texto en un vector de 3072 valores. En la ingesta se usa task_type = RETRIEVAL_DOCUMENT, porque se vectorizan fragmentos del documento.

```
response = client_genai.models.embed_content(
    model="gemini-embedding-001",
    contents=texto,
    config=types.EmbedContentConfig(task_type="RETRIEVAL_DOCUMENT"))
return response.embeddings[0].values
```

Paso 6. Ingesta: crear el índice vectorial

crear_indice_vectorial define en Atlas un índice de tipo vectorSearch sobre el campo embedding. Si el índice ya existe, se muestra un aviso y el proceso continúa.

Parámetro	Valor
name / type	vector_index / vectorSearch
path	embedding
similarity	dotProduct
numDimensions	3072 (debe coincidir con la dimensión del modelo de embeddings)

Paso 7. Ingesta: fragmentar e insertar

procesar_pdf divide el texto en trozos de 1000 caracteres, genera el embedding de cada uno e inserta todos los documentos con insert_many. Con el PDF de clase (aws-cloud-adoption-framework_XL.pdf) se insertaron 84 fragmentos.

```
trozos = [texto[i:i+1000] for i in range(0, len(texto), 1000)]
for i, chunk in enumerate(trozos):
    documentos.append({"id": i, "texto": chunk,
                       "embedding": crear_embedding(chunk)})
collection.insert_many(documentos)
```

Campo	Tipo	Descripción
id	int	Posición del fragmento dentro del documento.
texto	string	Contenido del fragmento que se enviará como contexto al LLM.
embedding	array (3072)	Vector usado por el índice para la búsqueda por similitud.

Paso 8. app.py: leer los secretos

La aplicación no contiene credenciales: las lee de st.secrets. La sección [app] corresponde a la estructura del archivo secrets.toml.

```
GOOGLE_API_KEY = st.secrets["app"]["GOOGLE_API_KEY"]
MONGODB_URI = st.secrets["app"]["MONGODB_URI"]
```

Paso 9. app.py: clientes en caché

Streamlit vuelve a ejecutar todo el script con cada interacción del usuario. Con @st.cache_resource, los clientes de Gemini y MongoDB se crean una sola vez y se reutilizan entre ejecuciones.

Paso 10. app.py: recuperación semántica

La pregunta se vectoriza con task_type = RETRIEVAL_QUERY y buscar_similares ejecuta un pipeline de agregación con $vectorSearch que devuelve los k fragmentos más cercanos y su score.

Parámetro	Valor / función
index / path	vector_index / embedding
numCandidates	100 candidatos evaluados antes de seleccionar los mejores
limit (k)	5 fragmentos devueltos
$project	Devuelve texto y score ($meta: vectorSearchScore)

Paso 11. app.py: generación de la respuesta

generar_respuesta une los fragmentos recuperados como contexto y construye un prompt que instruye a gemini-2.5-flash a responder exclusivamente con ese contexto e indicar cuando la respuesta no se encuentra en él.

```
Pregunta → embedding → $vectorSearch (top 5) → prompt + contexto → Gemini → respuesta
```

Paso 12. app.py: interfaz de chat

st.session_state.historial conserva la conversación; st.chat_input recibe la pregunta y st.chat_message muestra los mensajes. Un st.expander lista los fragmentos recuperados con su score, lo que permite verificar en qué parte del documento se basa cada respuesta.

Paso 13. Publicar el repositorio en GitHub

Cree un repositorio público con app.py y requirements.txt (streamlit, pymongo, google-genai, pandas). Nunca suba secrets.toml; si trabaja en local, agregue .streamlit/secrets.toml al .gitignore.

Paso 14. Desplegar en Streamlit Community Cloud con secretos

En share.streamlit.io seleccione Create app, elija el repositorio, la rama main y el archivo app.py. En Advanced settings → Secrets pegue la configuración siguiente y luego presione Deploy. Los secretos pueden editarse después desde Settings → Secrets de la aplicación.

```
[app]
GOOGLE_API_KEY = "<su_api_key>"
MONGODB_URI = "mongodb+srv://<usuario>:<password>@<cluster>.mongodb.net/"
```

Síntoma	Causa probable
KeyError en st.secrets	Secretos no configurados o sin la sección [app].
Timeout al conectar con MongoDB	Acceso de red no habilitado en Atlas (Network Access).
"No encontré información relevante…"	Índice no creado, aún en construcción o con numDimensions distinto.
ModuleNotFoundError	Falta la librería en requirements.txt.

Código fuente desarrollado en clase: app_llm.ipynb (ingesta) y repositorio chatbot-jcatg (app.py, requirements.txt), disponibles en CANVAS.

ACTIVIDAD PARA LOS ESTUDIANTES

- Cree su clúster siguiendo el manual "MongoDB Atlas" publicado en CANVAS.
- Ejecute la ingesta con el PDF de clase y verifique en Atlas la colección pdf_vectors y el índice vector_index en estado READY.
- Publique app.py y requirements.txt en un repositorio público de GitHub, sin credenciales.
- Despliegue la aplicación en Streamlit Community Cloud configurando GOOGLE_API_KEY y MONGODB_URI como secretos.
- Formule 5 preguntas (3 con respuesta en el documento y 2 sin ella) y registre la respuesta, los fragmentos recuperados y su score.
- Cambie k a 3 y a 10, y analice el efecto sobre la calidad de las respuestas.

TAREA EXTRA

Despliegue una segunda aplicación con un documento PDF de su interés (reglamento, manual técnico, informe sectorial, etc.):

- Realice la ingesta en una nueva colección para no mezclar fragmentos con el primer documento, y cree su índice vectorial.
- Ajuste en app.py el nombre de la colección, el título de la aplicación y el prompt según el dominio del documento.
- Use un nuevo repositorio público y una nueva aplicación en Streamlit Cloud con sus propios secretos.
- Entregue la URL pública de la aplicación y 3 capturas de interacciones.

PREGUNTAS DE ANÁLISIS

1. ¿Por qué no se envía el PDF completo al LLM y solo se recupera un subconjunto de fragmentos?
2. ¿Qué problema puede generar un chunking fijo de 1000 caracteres y cómo lo mejoraría?
3. ¿Por qué se usa RETRIEVAL_DOCUMENT en la ingesta y RETRIEVAL_QUERY en la aplicación?
4. ¿Qué ocurre si numDimensions del índice no coincide con la dimensión del embedding?
5. ¿Por qué las credenciales deben configurarse como secretos y no escribirse en el código del repositorio?
6. ¿Qué función cumple @st.cache_resource en una aplicación que se vuelve a ejecutar con cada interacción?
7. ¿La instrucción “usa EXCLUSIVAMENTE el contexto” elimina las alucinaciones? Justifique.
8. ¿Qué métricas usaría para evaluar la calidad del chatbot (por ejemplo, Recall@k o groundedness)?

ENTREGABLE SUGERIDO

Informe breve con: (1) URL pública de la aplicación y del repositorio; (2) capturas del clúster, la colección y el índice vectorial en Atlas; (3) captura de la configuración de secretos sin mostrar sus valores; (4) tabla de preguntas, respuestas y fragmentos recuperados; (5) análisis del parámetro k; (6) respuestas a las preguntas de análisis; y (7) evidencias de la tarea extra.

IDEA CLAVE

Un chatbot RAG no "aprende" el documento: lo consulta. La calidad de la respuesta depende tanto del LLM como de la recuperación (cómo se fragmenta, vectoriza e indexa el contenido), y la aplicación solo puede publicarse de forma segura cuando las credenciales se gestionan como secretos y no como parte del código.



Fecha límite de presentación: Revisar los detalles en CANVAS
Integrantes